Waley El-Nagdi

Software Developer

Backend Development & Systems Integration

Python · REST APIs · PostgreSQL · Business Applications

A self-taught developer applying for a company placement through the AQUA pathway toward the Lehrabschlussprüfung (LAP) in Applikationsentwicklung - Coding.

PERSONAL DETAILS

AGE
[redacted]

DATE OF BIRTH
[redacted]

LOCATION
Austria

EMAIL
waleyelnagdi7@gmail.com

PHONE
+43 676 5119613

FEATURED PROJECT

NexusOS

MULTI-TENANT PLATFORM

React SPA on an async FastAPI service layer with a PostgreSQL tenant model: authentication, RBAC, rate limiting and company-scoped data isolation.

DOMAIN nexuops.com
GITHUB github.com/waley7337

PYTHON · FASTAPI · POSTGRESQL · REDIS · REACT · TYPESCRIPT · DOCKER

5	7	~110	31
ARCHITECTURE LAYERS	SECURITY LAYERS	SECURITY TESTS	UI COMPONENTS

NEXT STEPS & ADDITIONAL LINKS: see final page

REDACTED COPY - date of birth and age removed; links to the private NexusOS repository replaced with the public GitHub profile; domain corrected to nexuops.com. The original document is unchanged and not published.

01 / WHO I AM

Professional introduction

I am a self-taught software developer. I have studied programming, application architecture and web technologies on my own, and I have turned that study into projects rather than keeping it theoretical. I am now looking for a company in Austria that will let me work in a real development environment through the AQUA pathway, with the goal of passing the formal **Lehrabschlussprüfung (LAP) in Applikationsentwicklung - Coding**.

Personal projects are one part of becoming a professional. The rest (working in an existing codebase, reviewing and being reviewed, testing, debugging real problems, documenting decisions and keeping version control clean) is learned in a team, and that is what I am asking for.

TARGET ROLE / POSITIONING

SOFTWARE DEVELOPER

Backend Development & Systems Integration

Python · REST APIs · PostgreSQL · Business Applications

AREAS OF INTEREST

Python and backend development · REST APIs and application architecture · web applications · CRM and ERP-style business software · database design · security and multi-tenant systems · automation and workflow applications

Personal statement

Software development is what I want to do, and I am excited by the thought of joining a team. My own projects have taught me how systems can be structured and where my understanding is still thin. I would rather learn those gaps from experienced developers, on real work, than keep guessing alone.

I bring a foundation I built myself, curiosity, steady learning habits and a willingness to work. What I need in return is mentorship, honest feedback and the chance to take on more responsibility step by step.

02 / HOW I LEARN AND BUILD

My development approach

I taught myself software development, and AI tools are part of that. I use large language models (LLMs) as learning assistants: to explain unfamiliar concepts, explore possible implementations, and help troubleshoot. I do not treat generated code as something to paste in and forget.

PRACTICE	WHAT IT MEANS IN PRACTICE
Understand first	I try to understand the code, the architectural choices and the trade-offs behind a solution before I rely on it.
Question the output	I treat generated answers as proposals to check, not as facts. Verification matters more than speed.
Investigate	I use the tools to dig into concepts I do not know yet, then work through the details myself.
Stay honest	I do not yet understand every component of my projects equally well. I am actively building that competence.

What I want from a professional environment is what this approach cannot supply on its own: code review, real constraints, and people who can tell me when my reasoning is wrong.

02 / HOW I LEARN AND BUILD

My Software Development Workflow

I follow a structured, iterative, verification-driven software development methodology:

01 Identify > 02 Analyze > 03 Design > 04 Implement > 05 Test > 06 Debug > 07 Verify > 08 Refine > 09 Document > 10 Deploy > 11 Monitor

STEP	WHAT I DO
1. Identify	Define the problem, functional requirements, constraints, and expected outcomes.
2. Analyze	Decompose complex problems into manageable components, identify dependencies, and assess technical risks.
3. Design	Plan the architecture, data flows, APIs, security boundaries, and implementation strategy.
4. Implement	Build incrementally using modular, maintainable code and established development practices.
5. Test	Validate functionality, integrations, edge cases, and expected system behavior.
6. Debug	Investigate errors, trace root causes, and resolve defects rather than merely treating symptoms.
7. Verify	Confirm expected behavior through manual checks, automated tests, logs, and appropriate validation tools. Verify both the individual component and its impact on the wider system.
8. Refine	Improve reliability, maintainability, performance, and security based on findings.
9. Document	Record architectural decisions, technical configurations, verification results, known limitations, and remaining risks.
10. Deploy	Release only after the required checks have passed and critical risks have been addressed.
11. Monitor	Observe the deployed system, detect unexpected behavior, and validate its continued reliability.

MY CORE PRINCIPLE: VERIFY BEFORE YOU DEPLOY

I don't deploy based on assumptions. I deploy based on evidence.

I do not consider an implementation complete simply because the code runs or appears to work. I verify its behavior manually and with appropriate tools, test relevant edge cases, and investigate uncertainties before proceeding.

This principle is particularly important in multi-tenant systems, where a defect in authorization or data isolation could expose sensitive information belonging to multiple companies. A single unverified assumption can have consequences far beyond the component being developed.

My objective is to eliminate avoidable uncertainty through systematic testing and verification. While no testing process can guarantee absolute certainty, I aim to establish the highest reasonable level of confidence before deployment, with particular attention to security, data integrity, access control, and unintended side effects.

My rule is simple: no deployment without verification. No assumption that something works simply because it appears to. If I cannot demonstrate that a critical requirement has been met, I investigate further before releasing it.

02 / HOW I LEARN AND BUILD

My Problem-Solving Framework

Throughout the software development process, I continuously ask myself five fundamental questions:

QUESTION	WHAT I ASK MYSELF
What?	What is the problem, what needs to be built, and what should the expected outcome be?
Why?	Why does this problem exist, why is this solution necessary, and what purpose does it serve?
How?	How does the system work, how should I implement the solution, and how can I make it reliable?
What if?	What could go wrong, what edge cases or security risks might arise, and how should the system handle unexpected behavior?
What will happen if…?	What will be the consequences of this decision, how will changes affect other components, and will the solution remain maintainable and scalable?

MY PRINCIPLE

I don't just ask how to make something work. I question why it needs to work that way, what could go wrong, and what impact my decisions will have on the entire system. This allows me to approach software development analytically, anticipate problems, and build more robust solutions.

03 / FEATURED PROJECT

NexusOS overview

NexusOS is a multi-tenant business platform and one of my central independent projects: a React single-page application on top of an async FastAPI service layer and a PostgreSQL tenant model. GitHub profile: github.com/waley7337

LAYER	TECHNOLOGY	ROLE IN NEXUSOS
Backend / API	Python, FastAPI, SQLAlchemy 2.0, Alembic	Thin routers, service layer, token validation and role / permission guards
Database	PostgreSQL 16	System of record; company-scoped tables with composite foreign keys
Cache / supporting service	Redis 7	Fixed-window counters for authentication rate limiting
Frontend	React 19, TypeScript, Vite, TanStack Query, Zustand	Route-guarded SPA with a bearer API client and single-flight token refresh
Delivery	Docker, GitHub Actions, Vercel config, nginx	Container image, CI on every push (lint, type checks, migrations, tests)

What is implemented

- **Authentication** is implemented end to end, frontend and backend: register, login, refresh with rotation, logout, protected profile, password reset, email verification, plus health and readiness endpoints.
- **Security controls** are wired in the codebase and covered by a dedicated test suite (see the security architecture below).
- **Dashboard**: the interface is complete and currently renders fixture data. A living **design system** route documents the 31 shared components.
- **Domain schema** for CRM, projects, finance, HR and inventory is modelled and migrated; live CRUD routes for these modules are not built yet.

Why separation matters in business software

Separating the interface, the API, the data layer and supporting services keeps each part replaceable and testable, limits the blast radius of a bug, and lets several developers work in parallel without stepping on each other. It is also what makes later security and scaling work possible. The next three pages show the system, security and multi-tenant architecture of NexusOS in full detail.

Cloudflare

EXTERNAL SECURITY PERIMETER · PRIMARY INTEGRATION

This layer sits in front of the application and protects it at the network and HTTP edge, before requests reach the backend.

INTENDED SECURITY FUNCTIONS

WAF and DDoS protection

Filtering malicious traffic and mitigating denial-of-service attacks.

Bot protection and traffic challenges

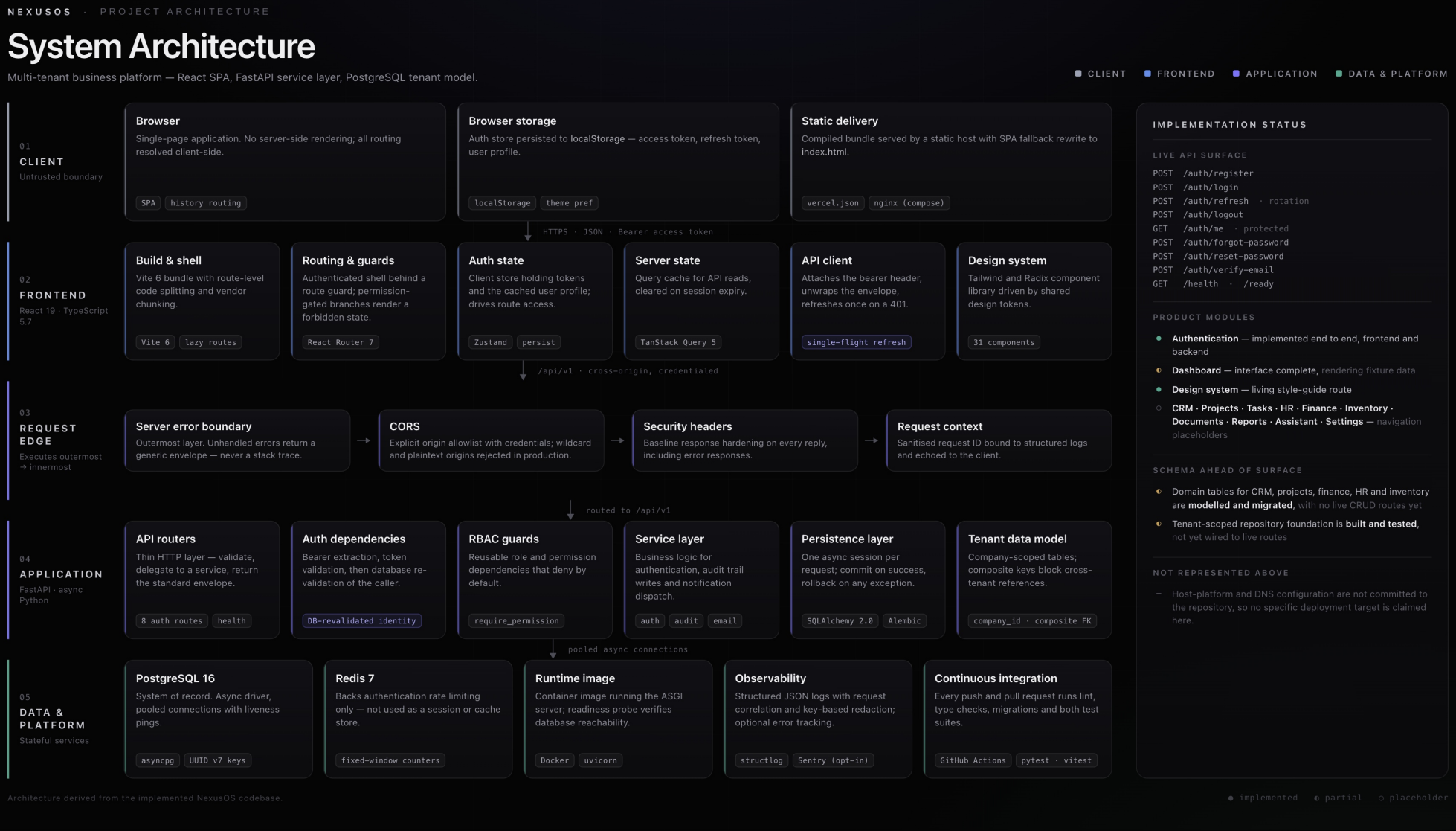
Detecting or challenging suspicious automated requests.

TLS and trusted-proxy handling

Protecting traffic in transit and ensuring the backend trusts client IP forwarding only from explicitly trusted infrastructure.

04 / NEXUSOS

System architecture



Five layers (client, frontend, request edge, application, and data & platform) from the untrusted browser to PostgreSQL and Redis. The status panel separates what is implemented (authentication, design system), partial (dashboard with fixture data) and placeholder (further product modules).

05 / NEXUSOS

Security architecture

NEXUSOS · PROJECT ARCHITECTURE

Security Architecture

Layered controls applied to every request, from untrusted client to tenant-scoped data.

LAYER 01
Client
Untrusted — treated as hostile

Nothing supplied by the browser is trusted for an authorization decision. Access tokens are **short-lived**, and a failed refresh tears down local state and forces re-authentication.

15-minute access token · single-flight refresh · session teardown on refresh failure · tokens held in browser storage

LAYER 02
Request boundary
Origin & response hardening

Cross-origin access is restricted to an explicit allowlist, and startup **refuses to boot in production** on a wildcard or plaintext origin. Baseline headers are applied to every response, including error responses.

CORS allowlist · credentialed · nosniff · frame denial · no-referrer · no-store · HSTS when TLS-terminated

LAYER 03
Client identity
Trusted-proxy boundary

Forwarded headers are **ignored by default** and honoured only when the immediate peer sits inside a configured trusted network. One resolver feeds rate limiting, transport-security decisions and the audit trail, so client IP cannot be spoofed independently.

forwarded headers untrusted by default · CIDR-scoped proxy trust · hop & header-size caps · universal ranges rejected

LAYER 04
Abuse control
Pre-authentication

Every credential-accepting endpoint is rate limited per resolved client address using shared counters, so limits hold across application instances.

6 auth endpoints limited · distributed fixed-window counters · keyed on resolved client IP · fails open if the limiter is unavailable

LAYER 05
Authentication
Credentials & sessions

Passwords are stored with a memory-hard hash. Tokens are signed with a **pinned algorithm** and their type is checked, so a refresh token can never be replayed as an access token. Refresh tokens rotate, and replaying a spent one **revokes every session** for that user.

Argon2 password hashing · algorithm-pinned JWT · token-type validation · refresh rotation · server-side session records · reuse detection → revoke all · non-enumerating errors

LAYER 06
Authorization
Re-derived, never assumed

A valid signature only proves the token was issued. Role and permissions are **re-read from the database on every protected request**, so suspending a user or deactivating a tenant takes effect immediately rather than when the token expires.

permissions read from database · deny by default · fails closed on suspension · fails closed on tenant deactivation · role · permission guards

LAYER 07
Tenant boundary & data
System of record

Tenant identity is taken from the **stored user record**, not from a token claim, header or parameter. The database independently rejects relationships that would cross a tenant, and all access goes through parameterised queries.

tenant derived server-side · composite foreign keys · parameterised ORM access · isolation violations logged critical · scoped repository not yet on live routes

SCOPE OF THIS DIAGRAM

Only controls present and wired in the codebase are shown. Controls that are absent or deliberately deferred are listed rather than omitted.

CROSS-CUTTING CONTROLS

- ✓ **Append-only audit trail** — actor, tenant, action, resolved IP and user agent recorded for registration, successful sign-in and token-reuse events
- ✓ **Log redaction** — credential-bearing keys scrubbed from structured logs and audit payloads
- ✓ **Uniform error envelope** — internals and stack traces never reach the client; API documentation disabled in production
- ✓ **Startup configuration validation** — production refuses debug mode, placeholder signing keys, and unsafe origin settings
- ✓ **Input validation** — schema-validated payloads with password strength and slug format rules
- ✓ **Secrets from environment** — no production secrets committed to the repository
- ✓ **Dedicated security test suite** — ~110 tests covering proxy trust, authorization, tenant isolation, redaction and audit

DELIBERATELY NOT CLAIMED

- ○ **Database row-level security** — no policies exist; isolation is enforced in the application and by foreign keys
- ○ **Content-Security-Policy** — deferred to the static host for a JSON API
- ○ **Issuer / audience claims** — not issued or validated
- ○ **Access-token revocation** — stateless until expiry, mitigated by database re-validation
- ○ **Account lockout** — rate limiting only
- ○ **Verified-email gate at sign-in** — verification exists but is not enforced

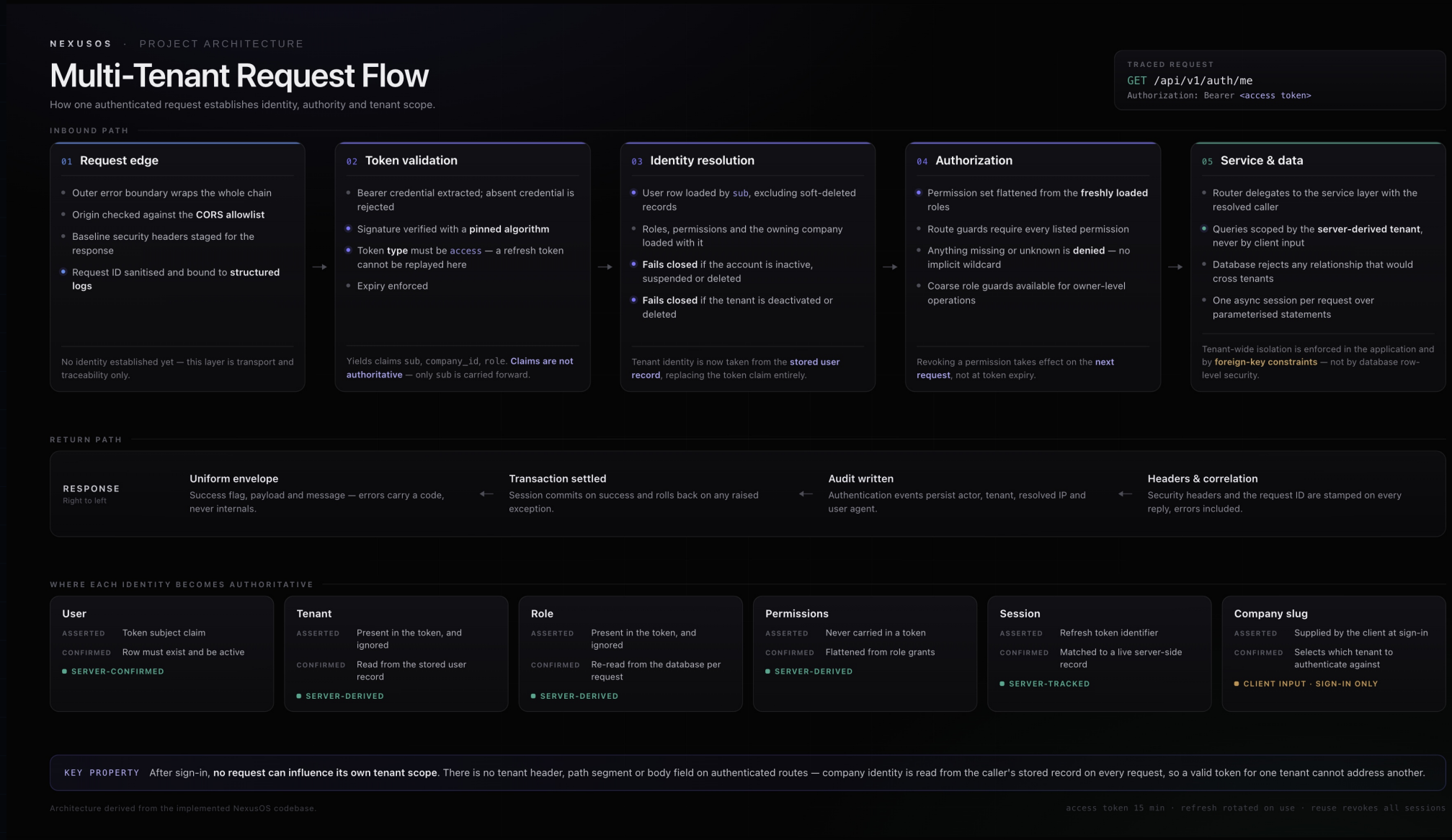
Architecture derived from the implemented NexusOS codebase.

defence in depth · fail closed on authorization · fail open on rate limiting

Seven layered controls from untrusted client to tenant-scoped data. Only controls present in the codebase are shown; absent or deliberately deferred ones (for example database row-level security and a Content-Security-Policy) are listed explicitly rather than omitted.

06 / NEXUSOS

Multi-tenancy and data isolation



Tenant identity is read from the caller's stored user record on every request, never from a header, path segment or body field, so a valid token for one tenant cannot address another. Isolation is enforced in the application and by foreign-key constraints, not by database row-level security.

07 / WHAT I WORK WITH

Technical skills and evidence

AREA	TECHNOLOGIES	EVIDENCE
Backend	Python, FastAPI, REST APIs, SQLAlchemy, Alembic	NexusOS repository
Data	PostgreSQL, Redis	NexusOS repository
Frontend	React, TypeScript, Vite, Tailwind	NexusOS repository
Security	JWT authentication, RBAC, rate limiting, tenant isolation	NexusOS repository and security test suite
Practices	Git / GitHub, CI with GitHub Actions, automated tests	Repository history and workflows

I list a skill only where I can show work that uses it.

08 / GOALS

What I want to learn and contribute

I WANT TO LEARN	I CAN CONTRIBUTE
• Working in an existing codebase • Code review, both giving and receiving • Testing and debugging real problems • Documenting decisions • Clean version control in a team • Delivering maintainable solutions	• Curiosity and independent learning habits • A self-built foundation in Python, APIs and web apps • Commitment and willingness to work • Openness about what I do not know yet • Interest in security and data isolation

09 / FOR EMPLOYERS

AQUA: A Practical Pathway to the Lehrabschlussprüfung

AQUA stands for *Arbeitsplatznahe Qualifizierung*, a workplace-based qualification route in the Austrian employment-support context. The intended arrangement combines practical learning in a company with coordinated theoretical or vocational training, set up together with the relevant **AMS** office and a training provider. The aim is to prepare me for the **Lehrabschlussprüfung (LAP) in Applikationsentwicklung - Coding** while I gain experience in a real company.

BENEFITS FOR THE COMPANY	BENEFITS FOR ME
• Evaluate my reliability, learning speed and technical ability in real work. • I learn your technologies, coding conventions, workflows and business domain. • Tasks can be introduced progressively, matched to my ability and the team's needs. • Develop a developer whose skills grow around your actual stack. • A structured framework for practical learning and formal qualification.	• Real-world coding experience. • Exposure to professional processes and existing codebases. • Mentorship, feedback and collaboration. • A link between independent learning and structured vocational training. • Preparation for the LAP in Applikationsentwicklung - Coding. • A chance to grow into a dependable developer over time.

Financial and administrative note

For the arrangement I am pursuing, the company contribution has been described to me as **approximately €500**. This is the figure stated for my proposed arrangement. It is not a universal price and not a guaranteed cost for every AQUA placement, and I am not claiming that it covers every expense or that the company has no further obligations.

Financial conditions, funding, responsibilities and eligibility must be confirmed with the AMS and the training provider before the arrangement begins. Within that framework, AQUA may give a company a comparatively accessible way to explore and develop talent, with the exact obligations set by the agreed training plan.

This is meant as a mutual arrangement with a clear learning structure and a formal qualification at the end of it.

10 / LET'S TALK

Contact and next steps

I would be glad to talk about a possible AQUA placement, to walk through NexusOS in more detail, and to answer questions about the pathway together with you, the AMS and the training provider.

CHANNEL	DETAIL
GitHub	github.com/waley7337
Location	Austria

NEXT STEPS

- Short introductory call or meeting.
- Technical walk-through of NexusOS and my learning approach.
- If there is mutual interest, clarify the AQUA conditions together with the AMS and the training provider.